Safe multi-user trading migration plan

Date: 5 October 2026 Status: planning only. No application implementation or live broker validation performed.

1. Outcome and scope

Multiple application users must connect their own Angel One accounts and run the existing strategies concurrently. One user's login, disconnect, session expiry, strategy control, orders, subscriptions, or broker errors must not change another user's account state or execution authority.

Preserve the familiar flow: register/sign in -> connect Angel One -> configure a strategy -> Start -> inspect position/status -> automatic or manual exit -> trade history. Preserve the existing strategy formulas, entry precedence, configured exit thresholds, trailing steps, quantity selection, PAPER pricing convention, API paths, and frontend layouts unless a separately documented safety correction requires different behavior.

Isolation means account-specific failures and actions stay within that account. Shared infrastructure can still fail; an absolute promise that no outage can ever affect multiple users is impossible. Production gates must demonstrate isolation and recovery, with explicit handling of shared database/service failure.

Scope starts with one linked Angel One account per application user. Use an internal brokerAccountId now so additional accounts can be supported later without weakening identity boundaries. Prevent two application users from independently controlling the same actual broker account. Resolve shared-account requests explicitly rather than creating duplicate owners.

2. Current workflow established from source

Stage	Current source behavior
App authentication	User records store hashed passwords. JWT protection resolves req.user. Registration creates the three strategy records.
Broker connection	React sends API key/client ID/MPIN/TOTP through authenticated Node login. Node sets a global active account; Python authenticates one global client and replaces/logs out the previous client. The form clears entered credentials.
Session/feed startup	Node starts Python readiness probes and periodically reads global session tokens. MarketDataService starts singleton broker market and order feeds. Python checks its single session in a background thread.
Start	Authenticated controller validates strategy configuration, checks the global active user, obtains the user/strategy runner, verifies broker status, attempts recovery, then starts discovery, candle evaluation, and scanner heartbeat tasks.
Discovery/data	Current index LTP determines option strike and expiry. Contracts refresh about every minute. Candle streams are reference counted, seeded from Python history, updated by broker ticks, and emit closed-candle events. HA warm-up loads prior trading-day data.
Signals	Registered backend plugins execute on candle events. HeikenAshi and ModifiedHeikenAshi calculate HA, EMA/JMA, wick/green filters, and two-red-candle reversal. CE takes precedence when both entry conditions qualify. Breakout evaluates a range and close breakout with buffer SL.
Entry	PAPER uses the supplied signal price. LIVE submits MARKET BUY, polls up to four confirmation observations, then stores position state, submits protective SL, records Trade, and starts one-second LTP monitoring.
Position management	LTP is fetched from Python. Breakout checks its range SL and entry-plus-target. Modified trailing mode checks final target and a stepped trailing SL. Other branches use entry-plus-target/entry-minus-SL. Candle evaluation can trigger reversal; manual exit has its own path.
Exit	LIVE cancels a stored protective order then submits MARKET SELL, listens for order updates, and uses an orderbook fallback after five seconds. Successful exits write P&L and clear position state. Failed exits can retry with capped exponential delay.
Cooldown	Breakout uses one minute; other strategies use four minutes for SL and two for target/trailing SL. Scanner resumes on later evaluation.
Status/history	React polls strategy status roughly every 1.5 seconds. Trade queries/statistics/overlays are generally filtered by authenticated user. Browser requests read status rather than owning the monitor.
Stop/restart	Stop currently clears monitoring and position references. Recovery currently occurs only during Start and does not restore all exit state. Process-local sessions and runners do not survive restart.

Stage	Current source behavior
Other pages	Manual terminal currently reads margins/index prices; its visible Place Order/Order Book buttons are not wired to handlers in the inspected component. Several strategy pages are placeholders. Migration must not claim these are functioning execution workflows.

Important baseline discrepancies: the breakout plugin reads candle.ts while normalized MarketDataService candles use time; several configuration fields accepted by validation/UI are not used by the active plugins; reversal evaluation is not gated by every displayed exit-mode choice. Freeze evidence of actual behavior and clarify intended behavior before changing those separately. Do not silently fix strategy semantics within account isolation work.

3. Shared-state inventory and disposition

Component	Current shared state / gap	Required disposition
activeAccountService	One module-level activeAccount and global active-user guard	Replace with verified user -> broker-account association and account-scoped availability.
brokerProxyController	Global account switch; status exposes global connection; login check and submission not one atomic operation	Per-account authenticated dispatch; serialize account connect/ reconnect; no cross-user account switching.
pythonApiClient and strategy HTTP helper	Most requests omit account context; background requests use global Python client	Mandatory verified account/ session context for all broker-dependent operations.
Python main/ broker_session	One angel instance, one client/ session ID/ user ID, one health thread	Isolated account workers/ sessions; strict request ownership; account-specific expiry and health.
smartStream/ orderUpdateService	One credentials/ JWT/ socket/ subscription set	One connection context per broker account/ session generation, with scoped events and cleanup.
MarketDataService	Global session flags/ promise/ fallback timers; tick cache keyed only by token; shared builders/ subscribers	Per-account context, exchange-aware instrument keys, scoped reference counts, timers and freshness.
StrategyEngineManager	Runners keyed user+strategy, but all use the global broker; session events broadcast to every runner	Bind user+account+strategy+mode; route session and order events only to their owner.
contractManager	Shared cache keyed underlying/ mode/ depth using global authenticated data	Account/ source-scoped computed results; immutable public instruments may be deliberately shared.
chartController/ candleService/ candleCache	Shared market caches and broker requests without owning-account dispatch	Authorize first; account-scope initial implementation. Shared public market data requires a separate entitlement/ source decision.
strategyStats	Per-user collectors plus global collector infrastructure	Preserve per-user reports; add account scope where relevant; admin aggregation explicitly authorized.
notifications	Global active-account fallback and one webhook destination	Explicit account context; owner-scoped delivery. Operations alerts need separately authorized destinations.
Python OrderStore	Shared local SQLite order attempts without ownership; global list/ clear helpers	Durable owner/ account-tagged attempts and restricted queries/ deletion. Internal-only routes are still isolation boundaries.
MongoDB models	User-owned strategies/ trades, but no stable broker account or complete execution snapshot	Add account identities, positions/ intents, settings and recovery state with owner-enforced repositories.
Frontend	Unscoped strategy localStorage keys; one origin JWT; global connection response	Scope settings by verified user/ account; clear state on identity change; prevent stale callbacks; do not reuse LIVE consent across users.
Rate limits	Per-user inbound limits, in-memory stores; background broker work is outside those limits	Per-account broker budgets, global resource budgets, distributed limits if multiple web instances.
Deployment	Process-local sessions and runners; startup probes don't recover positions	Explicit ownership/ routing, durable discovery and restart recovery; always-on services.

Component	Current shared state / gap	Required disposition
Safe globals	Plugin class registry, pure indicators, configuration, exchange schedule, immutable instrument catalog	May remain shared if no per-user mutable strategy instance or credential state is retained. Health/ warm-up can stay infrastructure-wide but cannot mark all accounts connected/ disconnected.

4. Target design and mandatory ownership boundary

Use shared React and Node API with a verified AccountContext: userId, brokerAccountId, brokerClientIdentity, sessionGeneration, executionMode, and ownerGeneration. Derive userId from authenticated identity; validate account ownership server-side. Browser headers/body IDs are selectors, never proof of permission.

Use an account supervisor owning one execution context and dedicated Python broker worker per active broker account for the initial small-user rollout. Workers have their own AngelClient, session health, market/order connections or explicitly routed Node feed context, retry state, and account-specific credentials. A private Python gateway/dispatcher maps the verified account to its worker; existing Node routes remain the browser interface. Do not accept browser-selected worker URLs or dynamic filesystem paths.

Keep feed adapters in their current Node layer initially, but instantiate them per account and session generation. Maintain one authorized trading owner per account across replicas. Python worker ownership/routing must be consistent across processes; simply adding Uvicorn workers would duplicate in-memory session registries. Resource limits and provisioning must be measured before picking worker counts.

This deployment choice can be replaced later with a proven session registry architecture. Do not combine that optimization with the initial migration. Every broker read/mutation, background task, recovery job and stream event must traverse an ownership boundary. Never fall back to whichever account happens to be connected when context is missing.

Secrets: MPIN/TOTP entered during connection are transient and never stored in plaintext or browser storage. If durable tokens are required, use encrypted storage with separately managed keys, least privilege and redaction. Session expiry/restart may require user reconnect under existing policy. Automatic credential persistence/relogin requires a separately designed security policy; do not introduce it implicitly.

5. Phased implementation

Every phase must pass BOTH the single-user baseline suite and relevant two-user suite before moving forward. New account execution stays disabled until all dependency gates pass. Changes below are proposed, not authorized live rollout.

Phase 0 — baseline, broker capabilities and compatibility contract

Change: add deterministic fixtures and ownership/API inventory; capture deployed revisions/topology and open orders/positions. Verify broker order status, cancellation, partial fills, account/API-key pairing, rate and connection limits, IP/deployment requirements and multi-client authorization with official broker documentation/support. Document signal/configuration discrepancies above and the nine known reliability issues.

Remain unchanged: application routes, UI controls, plugin formulas and live execution path during baseline collection. No production test orders.

Verify: single user completes all actually implemented flows with fixed candles/quotes; compare signal, side, quantity, thresholds, trailing updates, cooldown and history. Two mock accounts demonstrate existing isolation failures as explicit expected-to-fail cases; tests never exercise wrong-account live orders.

Rollback: remove diagnostic/test additions; no execution or schema migration yet.

Phase 1 — additive database ownership and durable records

Change: add BrokerAccount ownership/status/version, StrategySettings keyed by user/account/strategy, account identity on Trade, durable Position and OrderIntent records. Persist immutable entry-time mode/quantity/instrument/exit snapshot, remaining exposure, protection state, session/owner generation, quote/reconciliation health. Add unique intent IDs and conditional lifecycle updates; transactions only where supported and needed across documents.

Remain unchanged: completed historical records, existing user/trade IDs and route response fields. Existing account still runs through compatibility adapters; migrations do not rewrite P&L.

Verify: one-user queries/overlays/statistics match baseline; A cannot read/update/delete B's records even using known IDs. Repeated migration is safe. Audit legacy duplicates before indexes. Backfill account ownership only from verifiable evidence; uncertain open records block automated adoption.

Rollback: disable new-schema consumers, retain additive fields, restore backed-up metadata only through scoped procedures. Never drop execution evidence.

Phase 2 — Node authorization, account context and Python dispatch

Change: centralize account ownership checks in controllers/repositories; pass verified context through Python transport, chart/candle paths, orders and background tasks. Reject missing/mismatched account context. Preserve paths through default owned-account resolution. Scope status to the caller. Validate order ID ownership before cancellation and position ownership before exit. Serialize concurrent connect/reconnect for the same account; forbid duplicate owners of one broker account.

Remain unchanged: JWT login flow, existing route names/payload compatibility and positive single-account behavior. Do not remove the old guard before end-to-end context dispatch exists.

Verify: single-user contract tests for all endpoints, including profile/margins/positions/orderbook/history/charts; A supplies B's account/order/position IDs or forged internal context and is denied. Test simultaneous logins, expired JWT, internal-secret mismatch, cached response authorization and app logout.

Rollback: close multi-user admission first; revert only after all active accounts are safely drained/transferred. Never route multiple live accounts into the old global client.

Phase 3 — Python broker session and credential isolation

Change: introduce account worker supervisor and private dispatcher. Each worker owns one client and separate session generation, health, locks, tokens and shutdown. Account A reconnect/logout never calls B's client. Scope all Python endpoints, attempts, health and errors, including internal list/clear routes. Bound health-check resources; ensure a slow account cannot exhaust the shared gateway. Snapshot/verify session generation around operations; retiring a session cannot redirect an in-flight operation to another account.

Remain unchanged: validated broker order payloads, login input fields, normalized success/error schemas and existing user credential entry flow. No automatic MPIN/TOTP persistence.

Verify: one-account adapter matches baseline. Two accounts connect simultaneously with different fake credentials; profile, margins, price, order, logout and failure reach the correct client only. Kill A's worker while B trades; B continues. Restart gateway and a worker; unavailable sessions fail closed and require reconnect where needed. Test stale health-check completion after reconnect.

Rollback: prevent new connects, retain per-account worker routing for active exposure; retire workers only after reconciliation. A single global session is not a safe rollback target for two active accounts.

Phase 4 — market/order feed and cache isolation

Change: instantiate smartStream, orderUpdateService and MarketDataService per account/session. Scope builders, subscriptions, tick caches, disconnect timers, fallback, warm-up and reconnection. Event envelopes include account/session identity. Separate scanner/held-position references. Repair empty-symbol REST fallback. Scope computed contract/chart caches; classify public data separately before sharing. Use bounded, non-overlapping quote/session checks and account/global broker budgets.

Remain unchanged: indicator inputs for identical valid candles, subscribed instruments/timeframes, closed-candle processing, existing quote cadence where broker limits permit, and strategy calculations.

Verify: one-user candle/quote fixtures match baseline. A/B subscribe to the same token and different exchanges; stopping/unsubscribing A retains B. Drop A's stream or issue A 429/timeouts; B continues within its latency budget. Reject old-session events and wrong-account order events. Test actual backend candle-to-plugin formats; track semantic fixes independently.

Rollback: retain the account adapter boundary and active position subscriptions; downgrade features only within an account. Never merge account sockets or caches to recover service.

Phase 5 — strategy execution, order reconciliation and nine reliability fixes

Change: bind each runner to user/account/strategy/mode; move held-position ownership into a durable position manager. Route only matching session events. Use one execution coordinator across entry, automatic/manual/reversal exit, protective stop and trailing replacement; track ambiguous acknowledgments and partial fills. Remove generic Python placement retries after unknown submission. Persist intents before broker mutations; reconcile before retrying. Manual exit uses held identity. Continuously verify protective status and cancel/fill races. Preserve pending completion until DB writes succeed.

Remain unchanged: plugin calculations, CE/PE precedence, intended quantities, configured SL/target/trailing values, successful PAPER/LIVE behavior and cooldowns. PAPER never calls live mutations. Do not switch an open position's mode/quantity/settings when a new Start arrives.

Intentional corrections: Stop disables new entry scanning while existing positions remain managed; missing identity/unknown outcomes report reconciliation-required rather than false success; invalid/stale quotes show degraded monitoring. Label these changes clearly in the UI. They repair unsafe outcomes rather than silently changing signals.

Verify: baseline all three plugins and both modes. A target/manual/reversal exit and B trailing update concurrently must produce only account-owned orders. Test double Start, double Exit, stop during submitted BUY, delayed fills, partial fills, duplicate events, protection rejection/expiry, cancel/fill race, strike rotation and DB outage. Unknown mutations must not be blindly repeated. Existing manual broker routes must reconcile their effects with managed exposure.

Rollback: disable new entries for affected accounts while the compatible position manager manages existing exposure. Roll back execution code only after unresolved intents/positions are closed or transferred with verified ownership.

Phase 6 — startup, restart and session recovery

Change: after DB readiness discover all unresolved accounts/positions/intents, acquire conditional ownership, reconcile account-specific broker evidence, restore exact thresholds/trailing/instrument/mode/quantity, and resume management without enabling new entries. Repeat discovery after outages. Add account-scoped lease/owner generation and deploy handoff. New owners must reconcile old in-flight submissions; a lease alone cannot prevent a delayed old broker request from filling.

Remain unchanged: no new entry without the user's Start, existing reconnect credential policy and historical executions. Restart does not guess a missing broker account or synthesize zero P&L.

Verify: restart at each transition, before/after submission and fill; simultaneous old/new owners, late callbacks, Node/Python independent restarts, session expiry, database loss/restoration. A cannot adopt B's recovery record. Missing credentials leave that account visibly degraded; B remains operational if its dependencies are healthy. Force-kill recovery must use durable intents, not memory.

Rollback: recovery observation can be disabled while retaining current owners. Never remove a manager with unresolved exposure or move accounts to an unscoped recovery routine.

Phase 7 — frontend, settings, notifications and compatibility

Change: connection status belongs to the signed-in account; settings/local storage are user/account scoped, default LIVE consent safely on identity changes, clear old state and cancel/ignore old-user requests. Persist validated settings server-side. Keep same-origin shared JWT behavior explicit: different app users use separate browser profiles/devices; do not promise independent different-user tabs without redesigning session storage. Separate app logout from broker disconnect. Send owner-scoped notifications without global account fallback.

Remain unchanged: pages, navigation, connect form, strategy settings/control placement, existing routes and history filtering. Keep placeholders as placeholders. Preserve current APIs through additive fields/adapters; audit every caller.

Verify: single-user desktop/mobile workflow and response contracts. A then B in one browser cannot inherit A's trades/status/consent/settings. Two profiles run concurrently with independent margins and histories. Refresh/reopen never submits Start. App logout and browser closure preserve position management; broker disconnect explicitly degrades only that account. Failed logout cannot falsely imply broker protection exists.

Rollback: revert presentation using compatible status fields; retain account-scoped reads and storage. Never restore a global connection indicator for concurrent accounts.

Phase 8 — security, load, deployment and production release

Change: enforce TLS/private service authentication, secret redaction/encryption policy, audit logging with account identity, bounded per-account resource pools, account/global rate budgets, and always-on hosting. Separate infrastructure health from broker-account health. Configure external monitoring and graceful shutdown. Confirm broker approval/limits and IP/deployment requirements before admitting multiple live clients. Validate backup/restore and account-specific offboarding without deleting unresolved trades.

Remain unchanged: existing order authority requires authenticated owner and explicit LIVE configuration; release does not authorize production test orders or silently start strategies.

Verify: full single-user regression, 2+ user isolation matrix, auth penetration/negative tests, maximum expected user/strategy load, full-session soak, browser-free SL and target tests exceeding 30 minutes, worker/Node/DB failures and rolling deploy. Report quote gaps, owner age, pending orders, per-account rate use and exit submission latency separately from fill latency. Test shared-infrastructure outage as explicit degraded/recovery behavior.

Rollback: per-account feature/admission flags disable new entries; active compatible owners continue. Halt expansion on any cross-account action, false closure, unknown-order blind retry or missing owner. Retain additive schema and audit evidence. Use documented operator/broker procedures if the manager itself cannot protect exposure.

6. Nine-issue coverage and acceptance matrix

Prior issue	Phase(s)	Required result
Manual exit false success	2, 5	Held account/ instrument required; missing contract never marks LIVE closed.
Stop removes monitor	5, 7	Scanner stops; position owner/ subscription remains.
Incomplete restart recovery	1, 6	Exact durable execution snapshot recovered account by account.
Ambiguous/ duplicate submissions	1, 3, 5, 6	Durable intent and reconciliation before replacement; no generic mutation retry.
Assumed protective status	4, 5	Verified working quantity/ status; expiry/ reject/ fill/ cancel observed and reconciled.
Invalid REST fallback	4	Nonempty held symbol and validated fallback ticks; account-specific errors visible.
Overlap/ timeouts/ stale quotes	4, 5	Bounded per-position work, fresh valid data, old-generation callbacks discarded.
Contract rotation breaks reversal	4, 5	Held stream and reversal use held identity, independent of discovery.
Lost DB trade state	1, 5, 6	No silent completion, idempotent persistence/ recovery; no fabricated P&L.

7. Test requirements after every phase

Use deterministic mock broker clients with different account identities and deliberate overlapping order IDs/instrument tokens. Assert every outbound mutation's credentials/account/instrument/quantity, not merely HTTP success. Run through real Node -> Python dispatch in integration tests, not just isolated mocks of ownership helpers.

Each phase reruns existing backend tests, Python tests, frontend build/tests and applicable baseline fixtures. The phase-specific isolation tests above are mandatory. Tests run in isolated databases/workers and cannot select production broker credentials. Broker validation beyond mocks requires explicit live-test authorization and a bounded procedure.

Security matrix covers all read/write/delete endpoints, status, history export/overlays, settings, orders, positions, recovery discovery and internal Python routes. Test forged user/account/session selectors, stale sessions, duplicate client-account association, concurrent login, known foreign IDs and late background callbacks. Query scoping alone is insufficient if broker requests/events remain global.

Outage rules: before durable intent creation fails, block new entries/conflicting mutations and retain existing verified protection; alert urgently. After broker acceptance, reconcile the already-persisted intent and retain unknown state until evidence resolves it. The system cannot atomically transact MongoDB and the broker; do not promise exactly-once fills. Unresolved ambiguity must remain visible with a safe operator procedure.

8. Cutover and release evidence

1. Land baseline/schema/authorization in small reviewable changes with new execution disabled.
2. Observe account-scoped reconciliation read-only; it has no broker mutation authority.
3. Run single-user PAPER compatibility, then simultaneous multi-user PAPER and fault tests.
4. Resolve legacy live records and verify broker/deployment capability gates. Initial multi-user live cutover uses no unresolved legacy live positions/orders unless audited transfer is proven.
5. Admit a narrowly controlled first account, then a second after independent account evidence. Expand only within measured resource/broker limits.
6. Never operate old/global and new/account-scoped execution owners on the same position. Never fall back from a missing account context to the old global broker.

Production sign-off requires saved regression/isolation results, migration audit, ownership and restart evidence, broker contract matrix, capacity/soak report, secret-handling review, rollback rehearsal, and operator runbooks. The implementation is not production-ready simply because unit tests pass. Multiple user credentials alone do not authorize every broker/platform usage model; confirm the supported multi-client arrangement.

9. Source traceability and references

Reviewed workflow-bearing sources: backend index/config/routes/controllers/models/middleware; services strategyEngine, activeAccountService, marketDataService, smartStream, orderUpdateService, pythonApiClient, contractManager, candleService/cache, chartService, tradeOverlayService, strategyRegistry/stats, healthMonitor, pythonWarmup, notificationService and marketSessionManager; backend plugin/indicator implementations; Python main, broker_session, angel_client, order_store, instrument catalog and core configuration; frontend auth/router/connection form/API/polling, strategy pages, dashboard/manual terminal, charts and trades/history; existing tests and operations documentation.

Implementation must recheck exact file revisions before editing. Deployed account plans, broker capabilities and actual incident evidence remain unverified. Existing `SL_TARGET_RELIABILITY_IMPLEMENTATION_PLAN.md` supplies the reliability detail; this plan adds multi-user ownership and migration requirements and governs multi-user rollout.

Primary external references:

- https://smartapi.angelone.in/docs — broker contracts and capability validation.
- https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html — verified tenant context, ownership, cache/job isolation and quotas.
- https://www.mongodb.com/docs/v8.0/data-modeling/enforce-consistency/transactions/ — database atomicity and topology constraints.
- https://render.com/docs/free and https://render.com/docs/faq — inactivity limits and always-on compute behavior.

These references inform the proposed controls; they do not certify this application or guarantee exact-price execution/zero downtime.